



PURDUE POLYTECHNIC INSTITUTE
Department of Computer and Information Technology

CNIT 27200 – Spring 2025

Group Submission

Final Phase: Testing the Database

Due: Friday, April 25, 2025, 11:59pm via Brightspace

Instructions

- Test the database using assigned queries
- Document the project (see deliverables at the end of the document)
- Individual project reflection from each team member
- Evaluate the team performance

Test the Database

- ☐ During this phase of the project, your team will write and execute SQL statements to test the design and operation of the test database. The test data sets up a controlled environment for testing. Document all steps to make the query work. When building the queries, the use of the functions, filters, updates, etc. MUST be an appropriate use of the task. For example: do not use a COUNT function on a primary key. This would only give you a count of 1 for each item. This will result in a zero for the task.
- ☐ **Show all work.** The more details the better. If your team has to make a change to the database for a query to work appropriately, document this in the submission.
- ☐ Create a SQL statement and report the output for the following items in your team's DB (30):
 1. Start with a single table query with 4 attributes and one filter in the WHERE clause.
 2. In a single table query, filter between two dates. Include appropriate attributes (including the date attribute used in the filter) that would make the query useful.
 3. Use LIKE in a filter (with the % wildcard). Use LIKE in such a way that the filter cannot be accomplished with an equal sign. For example: where color like 'b%' to find all colors that start with b.
 4. In the select clause, calculate two attributes to make a temporary calculated column. Give it a column alias. For example: *Select price, quantity, price * quantity AS TOTAL_PRICE.*
 5. Use NOT IN to filter out a list of values from an attribute in a single table query.
 6. In the Select clause, use a Null Value function (NVL) on an attribute, and **right justify** another of your attributes in the same statement. Add # to fill the empty spaces in the right justified column, and choose an appropriate replacement for the null values.
 7. In your database, where you have a date stored (order date, event date, etc), find the **oldest** date along with the other fields in that table. (ie, oldest order in the ORDER table, oldest event stored in the DB).

8. Use the Count function to group by an attribute in your database. Use Count appropriately, as there should not be a count of 1 in each row of the result set. Find an attribute that uses the same value more than once (ie, do not use count on a primary key...).
9. Use the Sum function to add up a calculated column. For example: SUM(price*quantity) as TOTAL_ORDER.
10. Use the Average function on either a number or date in your database. Use Group by to group the average by one field in the result set. (ie, Average price of each equipment type)
11. Use HAVING to filter an aggregated function (use Count, Sum, Min, Max or Avg in a query)
12. Build a SQL statement that filters with **both** a WHERE clause and a HAVING clause.
13. Build an INNER JOIN between just two tables. The chosen attributes in the Select clause should come from both tables. Filter with at least two conditions in the WHERE clause.
14. Build a query that has to pull from at least 4 different tables. The attributes in the Select clause should come from at least two of the tables. Include a date as one of your attributes and set the date format to report out like: Thursday, December 2, 2021.
15. Use a nested subquery to find a pool of data and then the outer query is looking for records NOT IN the nested query. The inner query and outer query should be from different, but related, tables.
16. Use a subquery to find results that are greater than an average result in your nested query.
17. Build an INNER JOIN between two tables and include an aggregate function in the Select clause. Must also include the use of Group By.
18. Build a query that utilizes a LEFT OUTER JOIN. The result set should have parents that do not have children. (If you do not have this situation in your database, do an additional INSERT in the parent table in order to make the task work)
19. Build a query that utilizes a RIGHT OUTER JOIN. The result set should have children that do not have parents. (If you do not have this situation in your database, alter your database to allow a child table to have NULL values in the foreign key field. Then do an additional INSERT in the child table in order to make the task work).
20. Use UNION to join two tables together.
21. Use INTERSECT to find common values between two tables.
22. Make an update to a single record in one of your tables. Explain the update and then perform the task. Display the before, update, and after stmts/output.
23. Update a set of records in one statement (more than one row should be updated). Explain the update and then perform the task. Display the before, update, and after stmts/output.
24. Add 2 rows to one of your tables. Run a query to display the newly added rows. Explain the update and then perform the task. Display the before, insert, and after stmts/output.
25. Delete a row from one of your tables. Explain the deletion and then perform the task. Display the before, deletion, and after stmts/output.
26. Alter a table by adding a new column. Do a DESCRIBE after you have added the new column. Explain the update and then perform the task. Display the before, alter, and after stmts/output.
27. Populate the new column for all records in the table. It can be the same value for all records. Explain the insert and then perform the task. Display the before, insert, and after stmts/output.
28. Update a record using SUBSTR in the SET. Explain the update and then perform the task. Display the before, update, and after stmts/output. For example, update an area code in a phone number.

29. Create a view, an alternate key and an inversion entry for one of your tables. Run a query on the metadata to list the new view and indexes in your database.
30. Your choice. Build a query. Can be as simple or complex as you want. Run the query and display the results.

Final Project Deliverables (checklist) (45 points):

- ☐ Cover page with Project Title, Semester, Course, Date, and Your Team Members
- ☐ A pdf of your final ERD from your team project. Provide the most recent logical model and the physical model, print to PDF one model per page. It must be your latest version and each model on each page must be readable for credit.
- ☐ Resubmit the Select * results for all tables in your DB. This is helpful when grading in case your team made changes between the milestones. *You can submit the select * results file from Phase 2 with any updates that you may have made. I will use your script from Phase 2. If you made changes to your script, resubmit it with your final phase.*
- ☐ Enterprise Integration Plan – Using the Zachman model, provide a bulleted list of your implementation plan at the Planning Phase and the User Phase for each column in the model. No more than one page.
- ☐ Documentation of the test SQL statement execution and **formatted** output (sufficiently readable).
 - a. Name this document: **milestone_final.sql**. Create a .sql file with each of the SQL statements and the results. The sql statement should correspond with the question from the test list above. Use comments just as you did with Labs 3-8.
 - i. For each test query above:
 1. List the question # in comments
 2. list the team member responsible and the quality check team member
 3. Then the sql statement you created to fulfill the requirements of the question.
 4. Then the results in comments.
 5. Be as detailed as possible in each question.
 - b. **The test requirements are to be divided up among the team members.**
 - c. Even though one person is responsible, the entire team will be impacted by the work (if a query is incorrect, the team score is dropped as a whole).
 - d. Implement some review/confirmation process to assure accuracy. Document who was assigned and who completed the quality check for each query in the contribution chart and final sql document.
 - e. Be sure to include all required details per each question. Some of the questions require more than just the results. The more detail you provide will help with grading.
 - f. I will generate your ddl and dml Phase 2 script and then compare your sql statements to the script for accuracy.
- ☐ A minimum 300 word (single-spaced) reflection from each individual team member about the significance of the project and how it can be utilized beyond the classroom. You can self-reflect on the project – what were your obstacles, successes, etc. You can also infer the use of a database such as this in the organization.
- ☐ A zipped file on Brightspace with all the above files. Name your zipped file: **final_f21.zip**

Team Evaluations (out of 5 points) (If you don't submit any evaluations for your team members, you will lose 5 points. If you submit the evaluations for your team members, your grades for this part will be the average of teammates' evaluations for you.)

Each team member will be evaluated (confidentially) by the other team members. However, an individual score may be penalized if team participation, contributed material, and the Assessment Form evaluations warrant it.

The link to the Google Form (peer evaluation) is in Brightspace under the Project Final Phase Module.

Evaluations are to be submitted via Google Form by 11:59 p.m. on Friday, April 25th.